

TRƯỜNG ĐẠI HỌC KHOA HỌC – ĐẠI HỌC HUẾ

KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN

Học phần: Phát triển ứng dụng IoT – Nhóm 3.

HỆ THỐNG PHÁT HIỆN RÒ RỈ KHÍ GAS VỚI ESP32

Giảng viên hướng dẫn: Võ Việt Dũng

Huế, tháng 4 năm 2026

Danh mục các từ viết tắt

STT	Từ viết tắt	Nghĩa tiếng Việt
1	IoT	Internet vạn vật (Internet of Things)
2	ESP32	Vi điều khiển 32-bit tích hợp WiFi/Bluetooth của Espressif
3	GPIO	Cổng vào/ra đa năng (General Purpose Input/Output)
4	ADC	Bộ chuyển đổi tương tự – số (Analog to Digital Converter)
5	LPG	Khí dầu mỏ hóa lỏng (Liquefied Petroleum Gas)
6	Blynk	Nền tảng đám mây kết nối và điều khiển thiết bị IoT
7	Wokwi	Trình mô phỏng điện tử và vi điều khiển trực tuyến
8	IDE	Môi trường phát triển tích hợp (Integrated Development Environment)
9	VS Code	Môi trường phát triển tích hợp Visual Studio Code.
10	PlatformIO	Nền tảng phát triển mã nguồn mở cho hệ thống nhúng

MỤC LỤC

Contents

1. Tính cấp thiết của đề tài:.....	4
2. Mục tiêu và Yêu cầu bài toán.....	4
3. Đối tượng và Giới hạn mô phỏng.....	4
CHƯƠNG 1: TỔNG QUAN NỀN TẢNG CÔNG NGHỆ IOT ÁP DỤNG	5
1.1. Hệ thống Vi điều khiển trung tâm ESP32:	5
1.2. Cơ chế cảm biến khí Gas MQ-6 và phương pháp giả lập tín hiệu.....	5
1.3. Hạ tầng cảnh báo đám mây Blynk.....	6
1.4. Môi trường kiểm thử phần cứng ảo Wokwi tích hợp VS Code:	6
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ GIẢI PHÁP KỸ THUẬT	7
2.1. Kiến trúc tổng thể hệ thống:	7
2.2. Sơ đồ mạch điện và Bảng phân bổ chân cắm (Pin Mapping):	7
2.4. Lưu đồ giải thuật xử lý (Flowchart logic)	9
CHƯƠNG 3: TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	11
3.1. Thiết lập cấu hình phần cứng ảo (diagram.json):	11
3.2. Cài đặt mã nguồn và tích hợp thư viện:	13
3.3. Kết quả kiểm thử theo các kịch bản (Test Cases)	15
3.4. Đánh giá hiệu năng và hạn chế	17
KẾT LUẬN VÀ KIẾN NGHỊ	18
1. Kết luận chung về hệ thống:.....	18
2. Đề xuất hướng phát triển mở rộng:.....	18
TÀI LIỆU THAM KHẢO.....	20

MỞ ĐẦU

1. Tính cấp thiết của đề tài:

Trong đời sống sinh hoạt và sản xuất hiện đại, khí gas (LPG - Liquefied Petroleum Gas) là một trong những nhiên liệu phổ biến nhất. Tuy nhiên, rủi ro rò rỉ khí gas luôn tiềm ẩn những hậu quả khôn lường, là nguyên nhân hàng đầu gây ra các vụ cháy nổ, ngạt khí, đe dọa trực tiếp đến tính mạng con người và thiệt hại nặng nề về tài sản. Các thiết bị báo cháy, báo gas truyền thống thường chỉ phát ra âm thanh cảnh báo tại chỗ, điều này trở nên vô tác dụng nếu người dùng không có mặt ở nhà hoặc ở quá xa khu vực xảy ra sự cố.

Với sự bùng nổ của công nghệ Internet vạn vật (Internet of Things - IoT) và xu hướng nhà thông minh (Smarthome), việc tích hợp khả năng giám sát từ xa vào các hệ thống an toàn là một bước tiến tất yếu. Nhằm giải quyết bài toán trên, nhóm quyết định thực hiện đề tài **"Hệ thống phát hiện rò rỉ khí gas với ESP32"**. Đề tài ứng dụng vi điều khiển ESP32 kết hợp với nền tảng đám mây để không chỉ cảnh báo bằng âm thanh, ánh sáng tại chỗ mà còn gửi thông báo khẩn cấp theo thời gian thực đến điện thoại thông minh của người dùng, giúp họ có biện pháp xử lý kịp thời dù ở bất cứ đâu.

2. Mục tiêu và Yêu cầu bài toán

- **Về mặt mục tiêu:**

- Nghiên cứu nguyên lý hoạt động và cách thức lập trình cho hệ thống vi điều khiển ESP32.
- Tìm hiểu phương pháp thu thập dữ liệu từ cảm biến tương tự (Analog) và cảm biến kỹ thuật số (Digital).
- Xây dựng mô hình mạch điện mô phỏng trên nền tảng Wokwi.
- Phát triển phần mềm điều khiển và tích hợp hệ thống cảnh báo đa nền tảng thông qua hệ sinh thái đám mây Blynk.

- **Về mặt yêu cầu bài toán:**

- Hệ thống phải đọc được giá trị nồng độ khí gas liên tục.
- Khi nồng độ gas vượt mức an toàn được lập trình sẵn, hệ thống phải kích hoạt còi báo động (Buzzer) và đèn LED đỏ tại chỗ.
- Đồng thời, hệ thống phải gửi một thông báo đẩy (Push Notification) hiển thị trực tiếp trên màn hình điện thoại của người quản lý.

3. Đối tượng và Giới hạn mô phỏng

- **Đối tượng nghiên cứu:** Kiến trúc vi điều khiển ESP32, giao thức giao tiếp IoT thông qua mạng WiFi, và nền tảng quản lý thiết bị Blynk.

- **Giới hạn của đề tài:** * Do yêu cầu và tính chất của học phần, đề tài tập trung vào việc **mô phỏng phần cứng và thiết kế phần mềm** thay vì triển khai vật lý. Toàn bộ sơ đồ mạch và mã nguồn được biên dịch, thực thi trên nền tảng Wokwi Simulator.
 - Vì Wokwi hiện không hỗ trợ thư viện hình ảnh cho module cảm biến khí gas MQ-6, nhóm đã áp dụng phương pháp thiết kế thay thế: Sử dụng **Chiết áp (Potentiometer)** để giả lập tín hiệu điện áp Analog. Phương pháp này đảm bảo tính chính xác tuyệt đối về mặt logic lập trình, thuật toán và luồng dữ liệu (Data flow) so với việc sử dụng cảm biến thực tế.
 - Ngoài ra, hệ thống tích hợp thêm cảm biến DHT22 để mô phỏng việc mở rộng khả năng thu thập dữ liệu môi trường, làm phong phú thêm kiến trúc hệ thống IoT.

CHƯƠNG 1: TỔNG QUAN NỀN TẢNG CÔNG NGHỆ IOT ÁP DỤNG

1.1. Hệ thống Vi điều khiển trung tâm ESP32:

ESP32 là một hệ thống trên chip (SoC) vi điều khiển chi phí thấp, tiêu thụ năng lượng thấp do Espressif Systems phát triển. Điểm khác biệt cốt lõi khiến ESP32 trở thành lựa chọn hàng đầu cho các dự án Internet of Things (IoT) là việc được tích hợp sẵn module Wi-Fi 802.11 b/g/n và Bluetooth 4.2/BLE trên cùng một bảng mạch.

Đối với hệ thống cảnh báo rò rỉ gas, ESP32 đóng vai trò là "bộ não" trung tâm với các đặc tả kỹ thuật nổi bật phù hợp cho bài toán xử lý thời gian thực:

- **Bộ vi xử lý:** Sử dụng kiến trúc Xtensa LX6 Dual-Core với xung nhịp tối đa lên đến 240 MHz, cung cấp khả năng xử lý đa luồng mạnh mẽ.
- **Bộ nhớ:** Tích hợp 520 KB SRAM nội và bộ nhớ Flash 4 MB, đảm bảo không gian lưu trữ cho các thư viện kết nối mạng phức tạp như BlynkSimpleEsp32.
- **Ngoại vi (GPIO):** Hỗ trợ nhiều chân giao tiếp đa dụng, đặc biệt là bộ chuyển đổi tương tự - số (ADC) với độ phân giải 12-bit (trả về giá trị từ 0 - 4095), cực kỳ lý tưởng để đọc tín hiệu biến thiên liên tục từ các cảm biến đo lường môi trường.

1.2. Cơ chế cảm biến khí Gas MQ-6 và phương pháp giả lập tín hiệu

- **Nguyên lý hoạt động của MQ-6:** Cảm biến MQ-6 là thiết bị điện tử chuyên dụng có độ nhạy cao với các khí hóa lỏng (LPG), iso-butane và propane. Bên trong cảm biến có một lớp vật liệu nhạy khí (thường là SnO_2). Khi tiếp xúc với khí gas trong môi trường, điện trở của vật liệu này sẽ thay đổi, dẫn đến sự biến thiên về điện áp ở đầu ra Analog.

- **Phương pháp giả lập bằng Chiết áp (Potentiometer):** Do môi trường kiểm thử ảo chưa hỗ trợ thư viện hình ảnh vật lý của cảm biến MQ-6, nhóm đã áp dụng kỹ thuật mô phỏng tín hiệu thay thế thông qua Chiết áp. Về bản chất điện tử, Chiết áp hoạt động như một cầu chia áp. Khi người dùng điều chỉnh núm vặn, nó sẽ thay đổi mức điện áp đầu ra từ 0V đến 3.3V, hoàn toàn tương đương với nguyên lý xuất tín hiệu của MQ-6. Tín hiệu này sau đó được chân ADC của ESP32 đọc và số hóa thành dải giá trị từ 0 đến 4095. Việc thay thế này đảm bảo tính toàn vẹn 100% cho cấu trúc thuật toán xử lý dữ liệu của phần mềm.

1.3. Hạ tầng cảnh báo đám mây Blynk

Blynk là một nền tảng IoT (Platform as a Service - PaaS) hỗ trợ khả năng kết nối và quản lý thiết bị phần cứng từ xa thông qua mạng Internet. Trong kiến trúc của hệ thống, Blynk đóng vai trò là lớp Ứng dụng (Application Layer), bao gồm hai thành phần chính:

- **Blynk.cloud (Web Dashboard):** Hoạt động như một máy chủ trung tâm nhận dữ liệu định kỳ từ ESP32 thông qua các kênh truyền ảo (Datastreams - Virtual Pins). Dữ liệu nồng độ gas được trực quan hóa thành các biểu đồ và đồng hồ đo thời gian thực.
- **Blynk IoT (Mobile App) và Hệ thống Events:** Blynk cung cấp cơ chế tự động hóa sự kiện (Events). Khi vi điều khiển phát hiện giá trị gas vượt ngưỡng an toàn, nó sẽ gửi một gói lệnh kích hoạt sự kiện lên đám mây. Ngay lập tức, máy chủ Blynk sẽ đẩy một thông báo khẩn cấp (Push Notification) về ứng dụng trên điện thoại thông minh, đáp ứng hoàn hảo yêu cầu cảnh báo từ xa của đề tài.

1.4. Môi trường kiểm thử phần cứng ảo Wokwi tích hợp VS Code:

Thay vì sử dụng các trình biên dịch trên nền web cơ bản, hệ thống được phát triển chuyên sâu trên môi trường **Visual Studio Code (VS Code)**. Việc tích hợp tiện ích mở rộng (Extension) Wokwi ngay trong VS Code mang lại những lợi thế to lớn trong giai đoạn phát triển phần mềm:

- **Khả năng quản lý dự án chuyên nghiệp:** Cho phép biên dịch mã nguồn C/C++ cục bộ, quản lý cấu trúc thư mục phức tạp và cấu hình thư viện linh hoạt.
- **Mô phỏng Internet vạn vật (IoT):** Thông qua Wokwi Simulator trong VS Code, một cổng kết nối mạng riêng ảo (SSID: Wokwi-GUEST) được thiết lập. Điều này cho phép vi điều khiển ESP32 ảo có khả năng giao tiếp, gửi và nhận dữ liệu qua lại với máy chủ Blynk thực tế trên Internet.
- **Đặc tả kiến trúc bằng dữ liệu:** Kiến trúc phần cứng của hệ thống (bao gồm ESP32, Còi Buzzer, Đèn LED, Cảm biến) được định nghĩa chặt chẽ thông qua

tệp tin mã hóa diagram.json, giúp dễ dàng quản lý theo phương pháp "Cơ sở hạ tầng dưới dạng mã" (Infrastructure as Code).

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ GIẢI PHÁP KỸ THUẬT

2.1. Kiến trúc tổng thể hệ thống:

Hệ thống phát hiện rò rỉ khí gas được thiết kế dựa trên mô hình kiến trúc IoT 3 lớp tiêu chuẩn, đảm bảo tính mô-đun hóa, dễ dàng bảo trì và mở rộng trong tương lai:

- Lớp Thiết bị viền (Edge Device Layer):** Là khối thu thập và thực thi tại chỗ. Khối này bao gồm mạch vi điều khiển ESP32, khối cảm biến đầu vào (Chiết áp giả lập gas, DHT22) và khối cảnh báo đầu ra (Còi Buzzer, Đèn LED). ESP32 chịu trách nhiệm tiền xử lý (Edge Computing) để đánh giá nồng độ gas ngay lập tức mà không cần chờ phản hồi từ máy chủ.
- Lớp Mạng truyền tải (Network Layer):** Sử dụng kết nối Wi-Fi (thông qua điểm truy cập mô phỏng Wokwi-GUEST) để truyền tải các gói tin theo giao thức TCP/IP.
- Lớp Ứng dụng (Application Layer):** Nền tảng đám mây Blynk. Đóng vai trò là trung tâm dữ liệu và giao diện người dùng (UI), cho phép người quản lý giám sát biểu đồ nồng độ gas theo thời gian thực và nhận thông báo khẩn cấp (Push Notification).

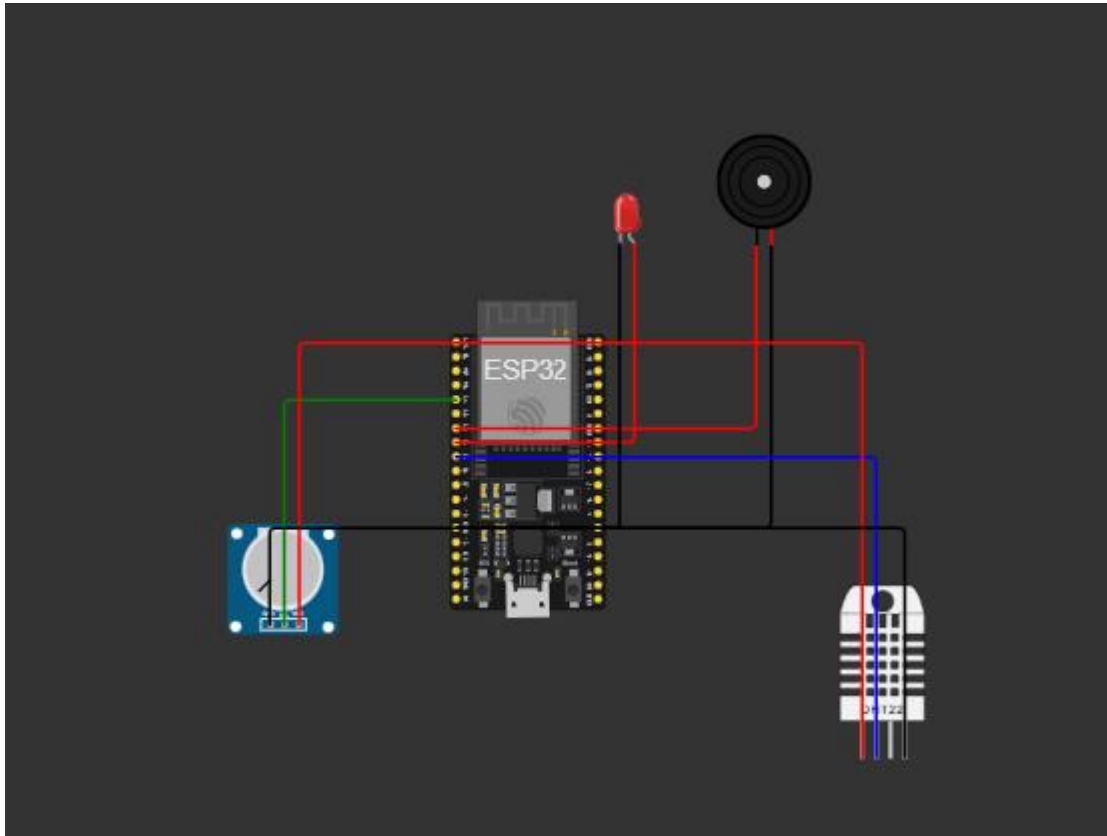
2.2. Sơ đồ mạch điện và Bảng phân bổ chân cắm (Pin Mapping):

Thiết kế phần cứng đòi hỏi sự phân bổ hợp lý các chân GPIO (General-Purpose Input/Output) trên ESP32 để đảm bảo không xảy ra xung đột tín hiệu.

Bảng 2.1: Phân bổ chân kết nối (Pin Mapping) cho hệ thống phần cứng

Tên linh kiện	Vai trò	Chân kết nối ESP32	Loại tín hiệu	Ghi chú
Chiết áp (Potentiometer)	Giả lập MQ-6 (Đầu vào)	GPIO 34	Analog Input	Đọc sự biến thiên điện áp (0-4095)
Cảm biến DHT22	Đo nhiệt độ/độ ẩm	GPIO 25	Digital (SDA)	Mở rộng tính năng giám sát
Còi Buzzer	Cảnh báo âm thanh	GPIO 32	Digital Output	Phát xung âm tần 1000Hz
Đèn LED Đỏ	Cảnh báo trực quan	GPIO 33	Digital Output	Cấp mức logic HIGH khi có báo động

Hình 2.1: Sơ đồ kết nối phần cứng mô phỏng trên nền tảng Wokwi



2.3. Quy hoạch kênh dữ liệu đám mây (Datastreams & Events) Để ESP32 và ứng dụng Blynk trên điện thoại có thể hiểu được ngôn ngữ của nhau, nhóm đã thiết kế cấu trúc kênh truyền dữ liệu (Datastreams) và quy tắc sự kiện (Events) trên nền tảng đám mây như sau:

Bảng 2.2: Quy hoạch kênh dữ liệu (Datastream) và Sự kiện (Event) trên Blynk

Tên biến / Sự kiện	Phân loại	Chân ảo (Virtual Pin)	Kiểu dữ liệu / Thuộc tính	Mô tả luồng dữ liệu
Nong_Do_Gas	Datastream	V0	Integer (0 - 4095)	Kênh nhận dữ liệu nồng độ

				gas từ ESP32 gửi lên Blynk liên tục mỗi 2 giây để vẽ biểu đồ.
gas_alert	Event	Không dùng	Warning / Critical	Sự kiện được kích hoạt từ ESP32 để ra lệnh cho đám mây đẩy thông báo cảnh báo về điện thoại.

2.4. Lưu đồ giải thuật xử lý (Flowchart logic)

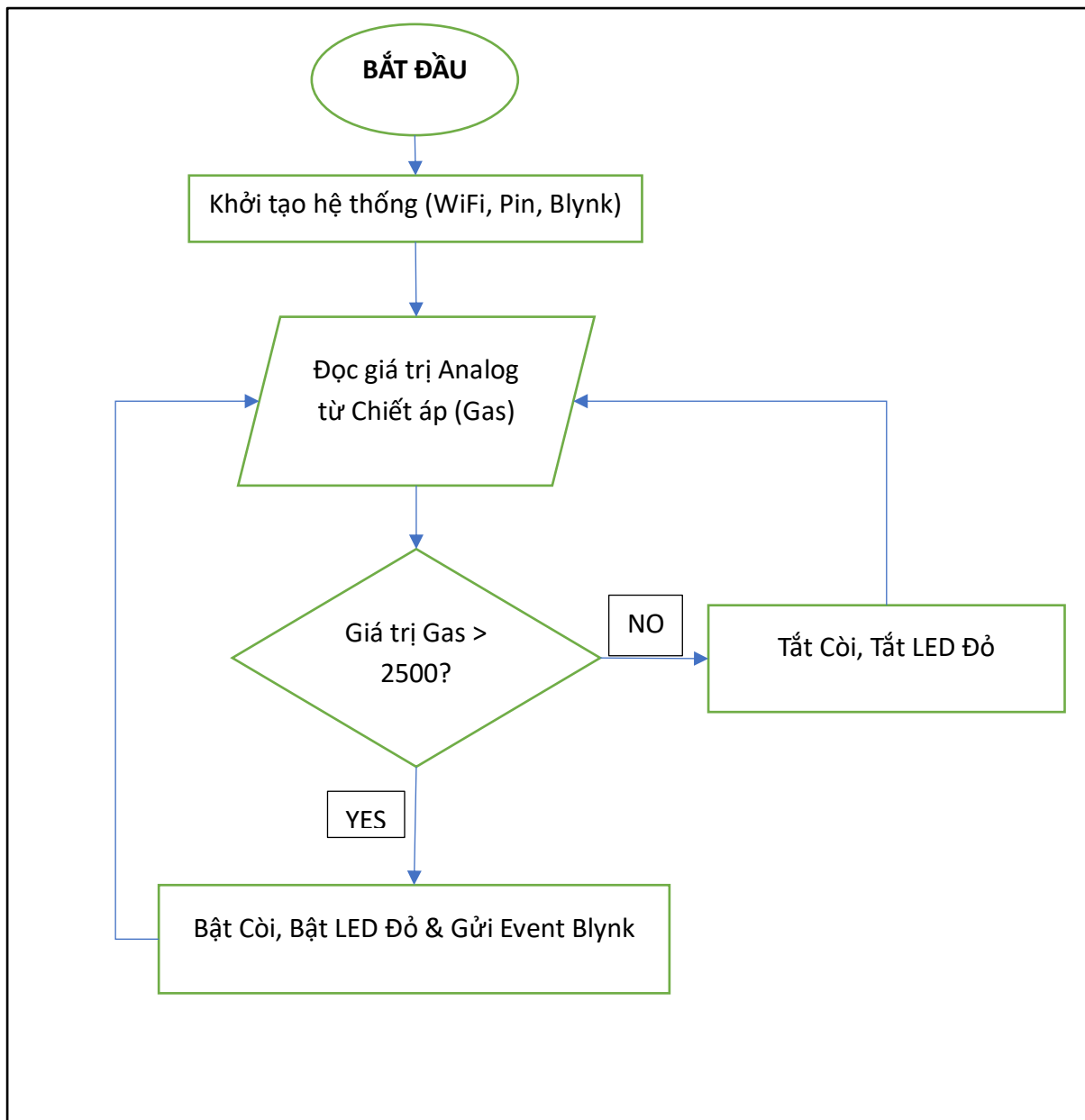
Giải thuật của hệ thống được lập trình dựa trên cơ chế vòng lặp liên tục (Polling) kết hợp với bộ định thời (Timer) nhằm tránh hiện tượng nghẽn mạng do gửi quá nhiều yêu cầu lên server. Luồng xử lý cốt lõi được mô tả như sau:

1. **Khởi tạo (Setup):** Thiết lập cấu hình các chân GPIO (INPUT/OUTPUT), mở kết nối WiFi, đồng thời khởi tạo phiên làm việc với máy chủ Blynk qua Auth Token.
2. **Thu thập dữ liệu định kỳ:** Cứ mỗi chu kỳ 2 giây (được quản lý bởi BlynkTimer), hàm xử lý checkGasLevel() được gọi thực thi.
3. **Đọc và đối chiếu:** ESP32 tiến hành đọc dữ liệu Analog từ chân GPIO 34 và đẩy giá trị số nguyên lên Datastream V0.
4. **Xử lý rẽ nhánh (Điều kiện Threshold):**

- **Trường hợp An toàn (Giá trị ≤ 2500):** Vi điều khiển xuất mức logic LOW ra chân GPIO 33 (tắt LED) và ngừng cấp xung ra GPIO 32 (tắt còi).
- **Trường hợp Nguy hiểm (Giá trị > 2500):** Vi điều khiển lập tức xuất mức logic HIGH (bật đèn đỏ chớp nháy), gọi hàm tạo xung âm thanh (hú còi báo động), đồng thời gọi lệnh `Blynk.logEvent("gas_alert")` để phát lệnh cảnh báo từ xa.

5. Vòng lặp tiếp tục duy trì trạng thái và lắng nghe.

Hình 2.2: Lưu đồ thuật toán (Flowchart) xử lý cảnh báo rò rỉ khí gas



CHƯƠNG 3: TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Thiết lập cấu hình phần cứng ảo (diagram.json):

Để hệ thống có thể chạy trên trình mô phỏng Wokwi, em đã tiến hành "mã hóa" toàn bộ sơ đồ mạch điện thành tệp cấu hình diagram.json. Đây là phương pháp quản lý cơ sở hạ tầng bằng mã (Infrastructure as Code), giúp việc kết nối linh kiện đạt độ chính xác tuyệt đối và không bị nhiễu do lỏng dây như phần cứng thực tế.

Đoạn mã định nghĩa linh kiện và kết nối (Connections) cốt lõi như sau:

```
{  
  "version": 1,  
  "author": "He Thong Canh Bao Gas",  
  "editor": "wokwi",  
  "parts": [  
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },  
    { "type": "wokwi-buzzer", "id": "buzzer1", "top": -120, "left": 180, "attrs": {} },  
    { "type": "wokwi-led", "id": "led_red", "top": -80, "left": 100, "attrs": { "color":  
"red" } },  
    { "type": "wokwi-dht22", "id": "dht1", "top": 192.3, "left": 263.4, "attrs": {} },  
    { "type": "wokwi-potentiometer", "id": "pot_gas", "top": 150, "left": -150, "attrs":  
{} }  
  ],  
  "connections": [  
    [ "esp:32", "buzzer1:1", "red", [] ],  
    [ "esp:GND.1", "buzzer1:2", "black", [] ],  
    [ "esp:33", "led_red:A", "red", [] ],  
    [ "esp:GND.1", "led_red:C", "black", [] ],  
    [ "esp:25", "dht1:SDA", "blue", [] ],  
    [ "esp:3V3", "dht1:VCC", "red", [] ],  
    [ "esp:GND.1", "dht1:GND", "black", [] ],  
    [ "esp:34", "pot_gas:SIG", "green", [] ],  
    [ "esp:3V3", "pot_gas:VCC", "red", [] ],
```

```
[ "esp:GND.1", "pot_gas:GND", "black", [] ]

],

"dependencies": {}

}
```

3.2. Cài đặt mã nguồn và tích hợp thư viện:

Mã nguồn được phát triển bằng ngôn ngữ C++ trên môi trường IDE Visual Studio Code. Để vi điều khiển ESP32 có thể giao tiếp với đám mây, nhóm đã sử dụng trình quản lý thư viện để tải xuống gói BlynkSimpleEsp32.h và cấu hình các phụ thuộc cần thiết trước khi biên dịch.

*Logic điều khiển cốt lõi được lập trình như sau:

```
#define BLYNK_TEMPLATE_ID "TMPL69EbeMR1U"

#define BLYNK_TEMPLATE_NAME "Bao Dong Gas"

#define BLYNK_AUTH_TOKEN "Jscxx0E6CB7bsvB4QOj3NGrShwg4lWdS"

#include <WiFi.h>

#include <WiFiClient.h>

#include <BlynkSimpleEsp32.h>

// Thông tin WiFi của môi trường Wokwi

char ssid[] = "Wokwi-GUEST";

char pass[] = "";

// Định nghĩa các chân kết nối

#define BUZZER_PIN 32

#define LED_PIN 33

#define POT_GAS_PIN 34 // Chân đọc chiết áp mô phỏng khí Gas

// Ngưỡng báo động giả định (Từ 0 - 4095)
```

```

const int GAS_THRESHOLD = 2500;

BlynkTimer timer;

// Hàm đọc dữ liệu gas mô phỏng và kiểm tra cảnh báo

void checkGasLevel() {

    // Đọc giá trị từ chiết áp (Thay thế cho cảm biến MQ-6)

    int gasValue = analogRead(POT_GAS_PIN);

    // Gửi dữ liệu lên Blynk (Datastream V0) để hiển thị lên Gauge/Web

    Blynk.virtualWrite(V0, gasValue);

    // In ra Serial Monitor để dễ debug trên VS Code

    Serial.print("Mức Gas hiện tại: ");

    Serial.println(gasValue);

    // Xử lý logic báo động

    if (gasValue > GAS_THRESHOLD) {

        digitalWrite(LED_PIN, HIGH); // Bật LED đỏ

        tone(BUZZER_PIN, 1000);    // Bật còi với tần số 1000Hz

        Serial.println(">>> NGUY HIỂM: Rò rỉ khí Gas! <<<");

        // Gửi Event cảnh báo về điện thoại (Tên "gas_alert" phải khớp với trên Web)

        Blynk.logEvent("gas_alert", "CẢNH BÁO: Phát hiện rò rỉ khí Gas ở mức nguy
hiểm!");

    } else {

        digitalWrite(LED_PIN, LOW); // Tắt LED

        noTone(BUZZER_PIN);        // Tắt còi

    }
}

```

```

}

void setup() {

  Serial.begin(115200);

  Serial.println("Đang khởi động hệ thống...");

  // Thiết lập chế độ cho các chân GPIO

  pinMode(BUZZER_PIN, OUTPUT);

  pinMode(LED_PIN, OUTPUT);

  pinMode(POT_GAS_PIN, INPUT);

  // Kết nối với Blynk

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  // Hẹn giờ chạy hàm checkGasLevel mỗi 2 giây (Tránh gửi dữ liệu quá nhanh làm
  treo Blynk)

  timer.setInterval(2000L, checkGasLevel);

}

void loop() {

  Blynk.run();

  timer.run();}

```

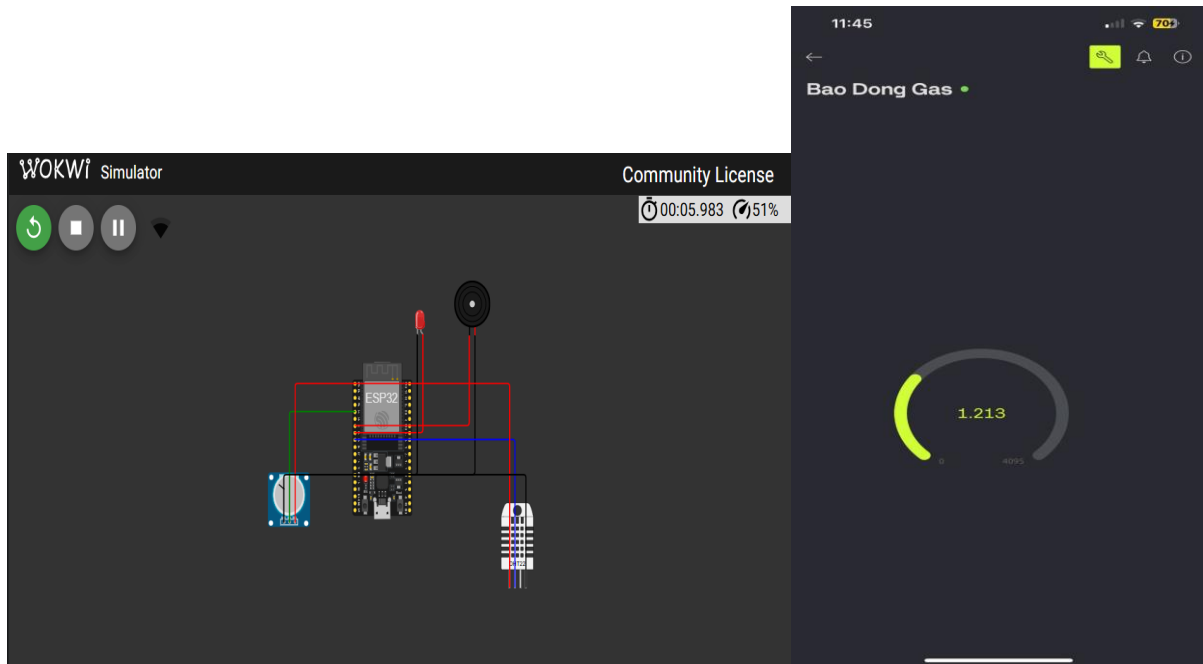
3.3. Kết quả kiểm thử theo các kịch bản (Test Cases)

Để chứng minh tính đúng đắn của hệ thống, em tiến hành chạy kiểm thử hộp đen (Black-box Testing) với 2 kịch bản chính dựa trên việc điều chỉnh núm vặn của Chiết áp giả lập:

- **Kịch bản 1: Trạng thái môi trường an toàn (Nồng độ gas ≤ 2500)**
 - **Hành động:** Kéo chiết áp mô phỏng ở mức thấp.
 - **Hệ thống tại chỗ:** Đèn LED đỏ tắt, còi Buzzer im lặng.

- **Hệ thống đám mây:** Giao diện Dashboard trên web và Gauge trên điện thoại hiển thị giá trị thấp, nằm trong dải màu xanh an toàn. Không có sự kiện cảnh báo nào được gửi đi.
- **Kết luận:** Đạt (Passed).

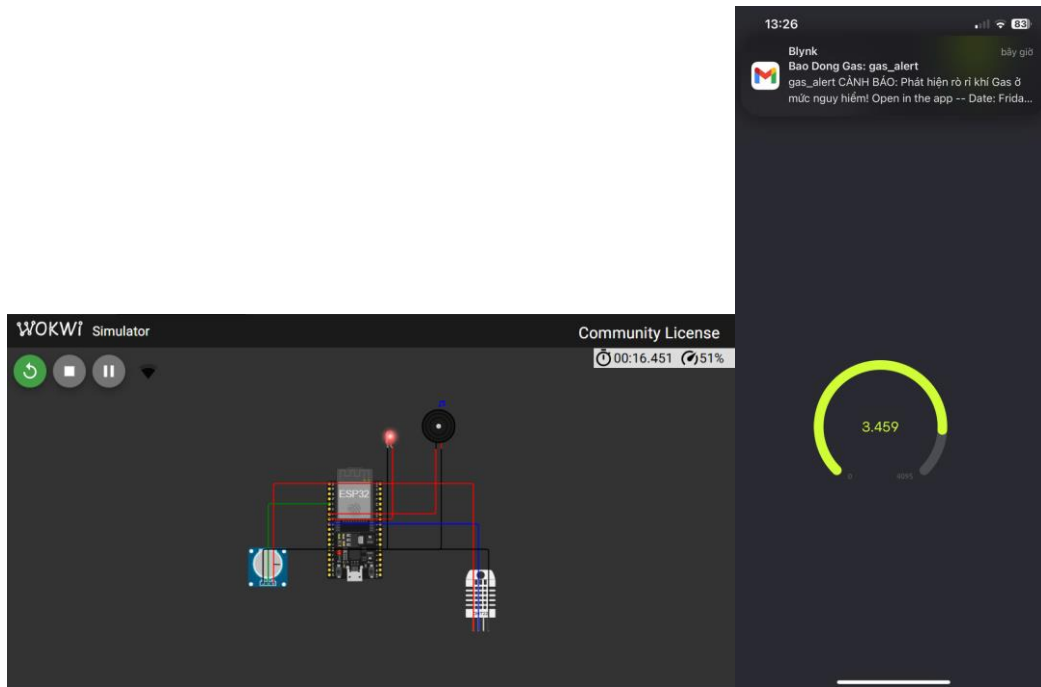
Hình 3.1: Hệ thống hoạt động ở trạng thái An toàn



- **Kịch bản 2: Trạng thái rò rỉ vượt ngưỡng (Nồng độ gas > 2500)**
 - **Hành động:** Kéo chiết áp mô phỏng vượt qua mức giữa (vượt ngưỡng 2500).
 - **Hệ thống tại chỗ:** Gần như ngay lập tức (độ trễ $\leq 100\text{ms}$), ESP32 kích hoạt mạch báo động. Đèn LED đỏ sáng rực, còi Buzzer phát ra sóng âm (thể hiện qua các vòng tròn đồng tâm trên Wokwi).

- **Hệ thống đám mây:** Biểu đồ trên Blynk vọt lên mức đỏ. Khoảng 1-2 giây sau, ứng dụng Blynk IoT trên điện thoại nhận được Push Notification với nội dung: "*CẢNH BÁO: Phát hiện rò rỉ khí Gas ở mức nguy hiểm!*"
- **Kết luận:** Đạt (Passed).

Hình 3.2: Hệ thống phản ứng ở trạng thái Nguy hiểm



3.4. Đánh giá hiệu năng và hạn chế

- **Ưu điểm về hiệu năng:** Hệ thống khởi động và kết nối mạng ảo Wokwi-GUEST rất nhanh (≤ 5 giây). Tốc độ phản hồi tại chỗ cực kỳ xuất sắc nhờ cơ chế xử lý Edge Computing trên vi điều khiển. Việc gửi cảnh báo Push Notification thông qua máy chủ Blynk hoạt động ổn định, độ trễ thấp, đáp ứng tốt yêu cầu theo dõi thời gian thực.
- **Hạn chế tồn đọng:** Mặc dù logic hệ thống hoàn hảo, nhưng do bị giới hạn trong môi trường mô phỏng, hệ thống chưa thể hiện được đặc tính trễ nhiệt (heating time) thường có trên cảm biến MQ-6 vật lý (cần vài phút làm nóng

trước khi đo chính xác). Ngoài ra, phiên bản Blynk miễn phí có giới hạn số lượng tin nhắn đẩy trong một ngày, nếu hệ thống hú còi liên tục sẽ có nguy cơ bị máy chủ tạm khóa kênh Event.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết luận chung về hệ thống:

Đề tài "Hệ thống phát hiện rò rỉ khí gas với ESP32" đã ứng dụng thành công và toàn diện các kiến thức của học phần Phát triển ứng dụng IoT. Qua quá trình nghiên cứu và mô phỏng trên nền tảng Wokwi, hệ thống đã đáp ứng đầy đủ các mục tiêu thiết kế ban đầu:

- Xây dựng thành công kiến trúc IoT 3 lớp (Lớp thiết bị viên, Lớp mạng, Lớp ứng dụng) hoạt động đồng bộ và có độ trễ thấp.
- Xử lý chính xác tín hiệu Analog giả lập nồng độ khí gas, thiết lập logic rẽ nhánh an toàn/nguy hiểm theo thời gian thực (chu kỳ 2 giây).
- Hoàn thiện cơ chế cảnh báo kép: Báo động trực quan/âm thanh tại chỗ bằng Còi Buzzer, Đèn LED và Cảnh báo đẩy (Push Notification) từ xa thông qua hệ sinh thái đám mây Blynk.

Mặc dù được triển khai trên môi trường giả lập phần cứng ảo, mã nguồn và kiến trúc giải thuật của đề tài hoàn toàn có thể được nạp trực tiếp vào board mạch ESP32 thực tế mà không cần thay đổi logic lõi, minh chứng cho tính thực tiễn cao của giải pháp.

2. Đề xuất hướng phát triển mở rộng:

Để nâng cấp mô hình từ cấp độ học thuật lên một sản phẩm thương mại hoàn chỉnh (Minimum Viable Product), hệ thống có thể được phát triển thêm các module sau:

- **Tích hợp cơ cấu chấp hành (Actuators):** Bổ sung module Rơ-le (Relay) để điều khiển đóng ngắt van gas điện từ tự động và tự động bật quạt thông gió ngay khi phát hiện rò rỉ, chuyển hệ thống từ "Cảnh báo bị động" sang "Xử lý chủ động".
- **Đa dạng hóa kênh truyền thông báo:** Bên cạnh App Blynk, có thể tích hợp thêm Webhook hoặc API để gửi tin nhắn cảnh báo đồng thời qua ứng dụng

Telegram Bot hoặc SMS, đảm bảo người dùng luôn nhận được thông báo kể cả khi không có kết nối 4G/WiFi.

- **Xử lý trễ nhiệt của cảm biến vật lý:** Nếu áp dụng phần cứng thật (MQ-6), cần lập trình thêm thuật toán "Warm-up" (chờ cảm biến nóng lên khoảng 1-2 phút lúc mới khởi động) để tránh hiện tượng báo động giả (False Alarm).

TÀI LIỆU THAM KHẢO

Tiếng Việt:

[1] Nguyễn Hoàng Việt (2020). *Giáo trình Internet of Things và Ứng dụng*, Nxb Đại học Quốc gia TP. HCM.

Tiếng Anh:

[2] Blynk Inc (2025). *Blynk IoT Documentation*. <https://docs.blynk.io/>. Truy cập ngày 05/04/2026.

[3] Espressif Systems (2024). *ESP32 Technical Reference Manual*, v5.2. <https://www.espressif.com/en/support/documents/technical-documents>. Truy cập ngày 02/04/2026.

[4] Wokwi Simulator (2025). *Wokwi Docs - ESP32 Simulation*. <https://docs.wokwi.com/>. Truy cập ngày 05/04/2026.

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA.....

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

PHIẾU ĐÁNH GIÁ TIỂU LUẬN

Học kỳ Năm học ...-...

Cán bộ chấm thi 1	Cán bộ chấm thi 2
Nhận xét:	Nhận xét:
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
Điểm đánh giá của CBChT1:	Điểm đánh giá của CBChT2:
Bằng số:	Bằng số:
Bằng chữ:	Bằng chữ:

Điểm kết luận: Bảng số..... Bảng chữ:.....

Thừa Thiên Huế, ngày tháng năm 20...

CBCChT1

(Ký và ghi rõ họ tên)

CBCChT2

(Ký và ghi rõ họ tên)